

End-to-End DevOps CI/CD Project using Terraform, Jenkins, SonarQube, Trivy & Docker

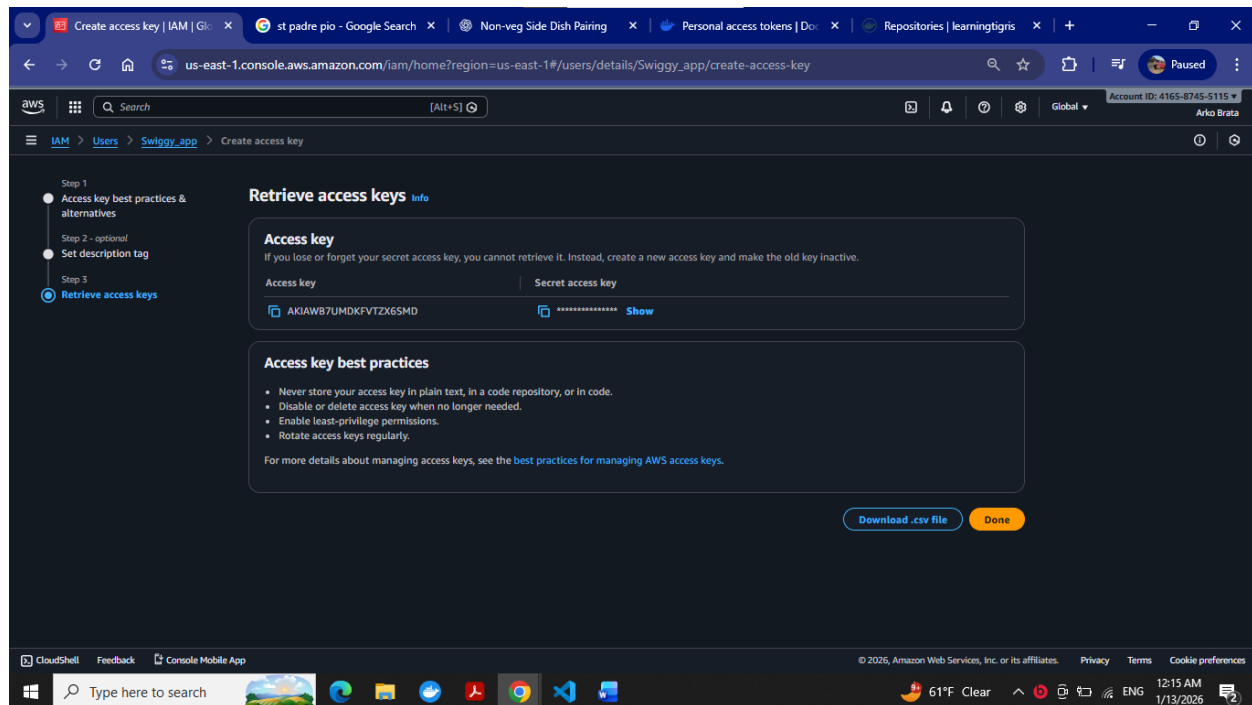
Project Overview

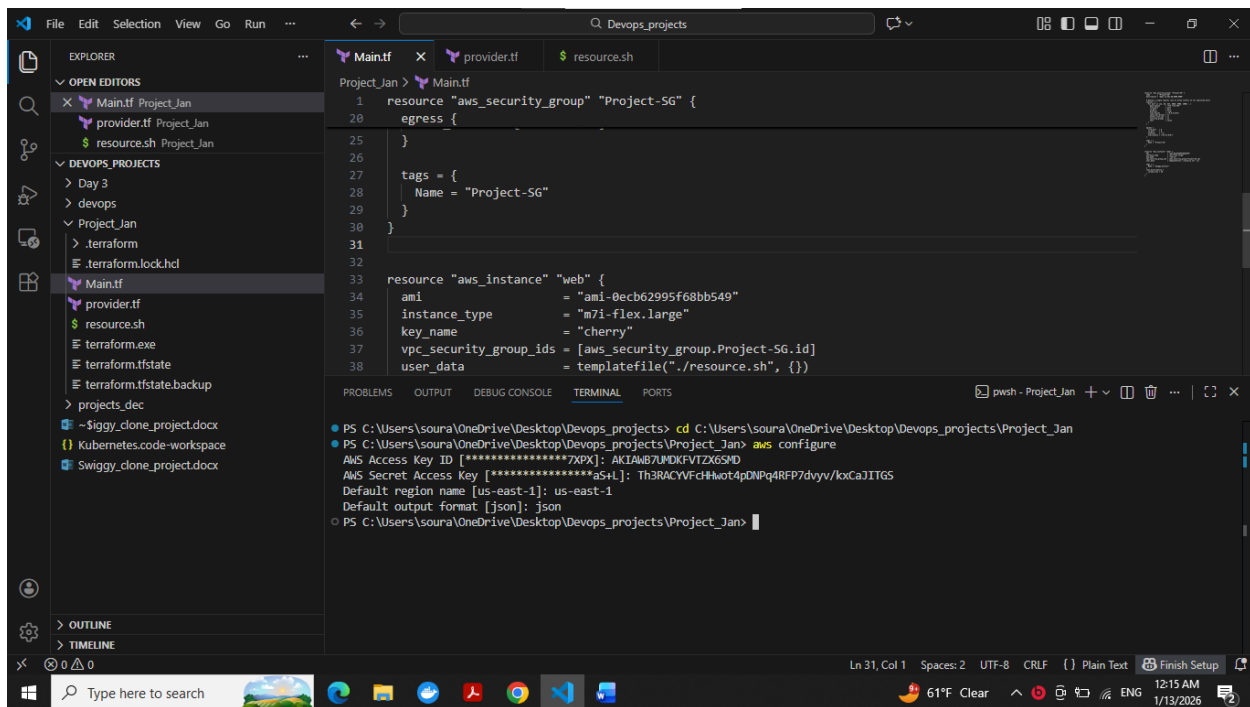
This project demonstrates a complete end-to-end DevOps CI/CD pipeline where infrastructure is provisioned using Terraform, application code is managed through GitHub, continuous integration and deployment are handled by Jenkins, code quality is enforced using SonarQube, security vulnerabilities are detected using Trivy, and the application is containerized and deployed using Docker on AWS EC2.

Step 1: Provision AWS Resources with Terraform

AWS Account Configuration:

- Created an IAM user with programmatic access.
- Generated **Access Key** and **Secret Access Key**.
- Configured AWS CLI on the EC2 instance using:
aws configure
- Provided access key, secret key, region, and output format.





Terraform Source Code

GitHub Repository:

<https://github.com/learningtigris/Terraform-Script-Swiggy.git>

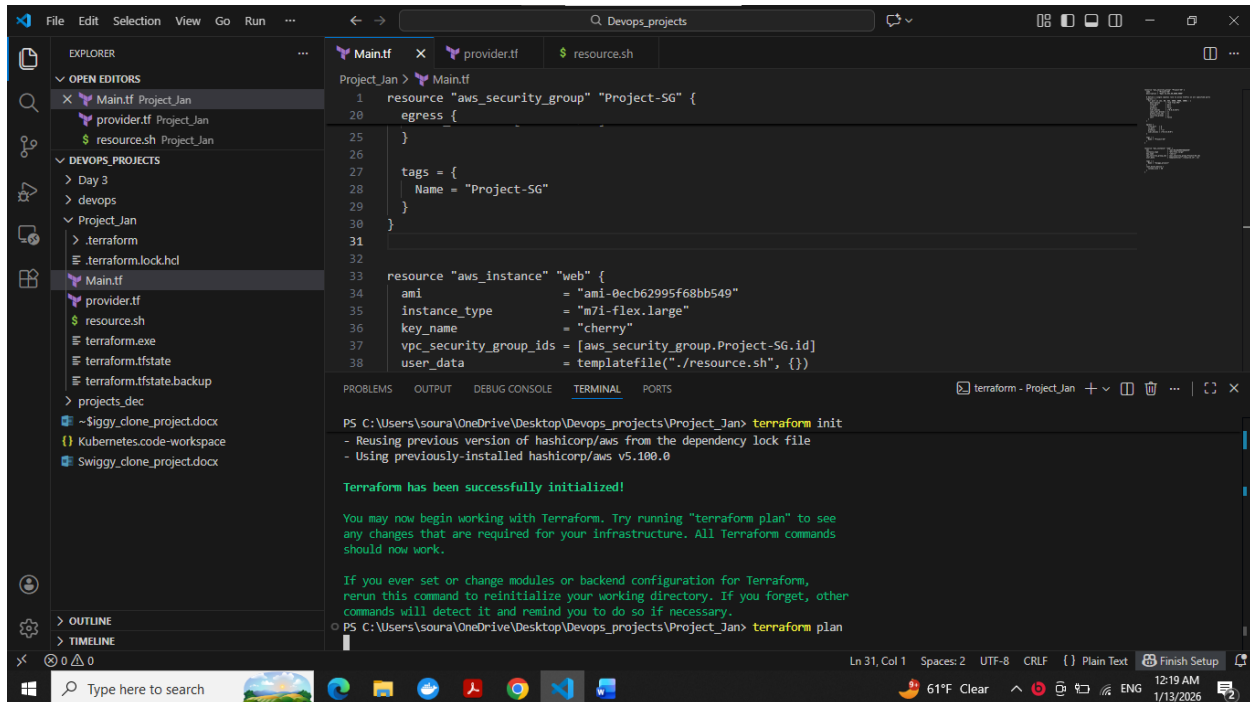
Key Files:

- **main.tf** → Defines Terraform configuration blocks and resources.
- **provider.tf** → Configures AWS provider and region.
- **resource.sh** → Shell script to install: Java, Jenkins, Docker, SonarQube, Trivy.

Terraform Execution flow:

terraform init

Initializes provider plugins



The screenshot shows the Visual Studio Code interface with the Explorer view on the left displaying the project structure. The main editor shows the `Main.tf` file with Terraform configuration for an AWS security group and an instance. The terminal at the bottom shows the execution of the `terraform init` command, which successfully initializes the Terraform environment and lists the providers used.

```
PS C:\Users\soura\OneDrive\Desktop\Devops_projects\Project_Jan> terraform init
- Reusing previous version of hashicorp/aws from the dependency lock file
- Using previously-installed hashicorp/aws v5.100.0

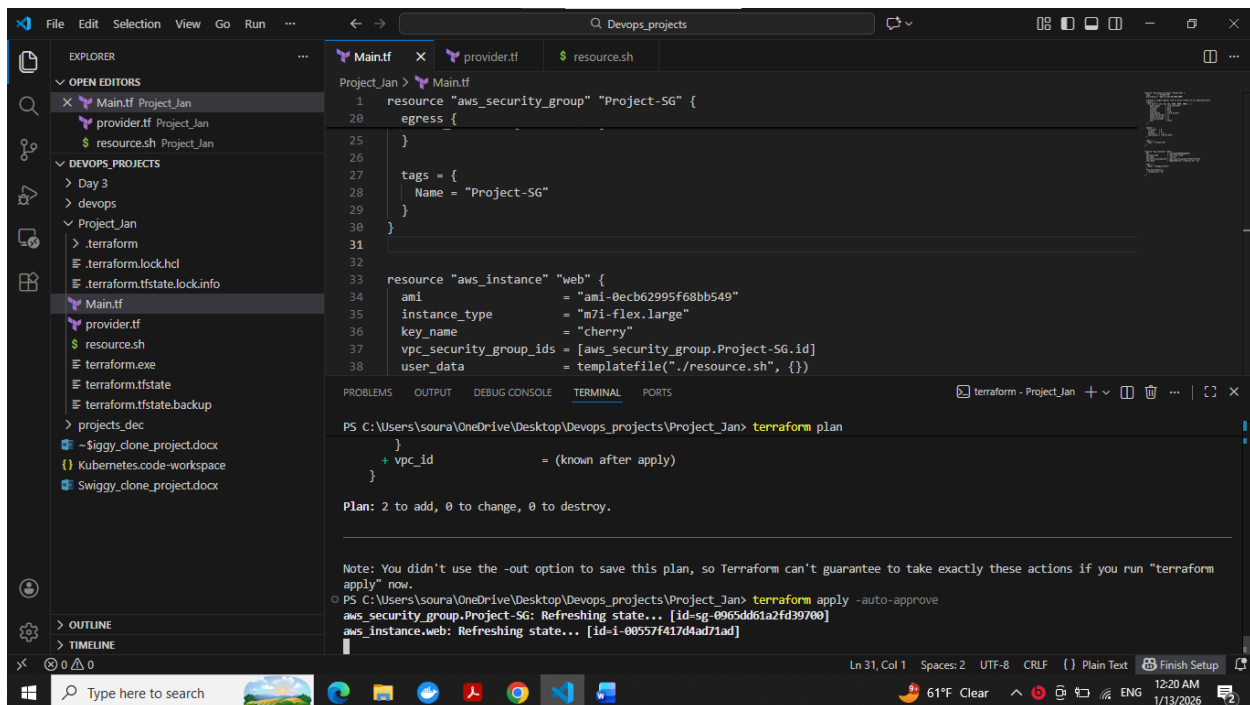
Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
PS C:\Users\soura\OneDrive\Desktop\Devops_projects\Project_Jan> terraform plan
```

terraform plan

Shows infrastructure execution plan



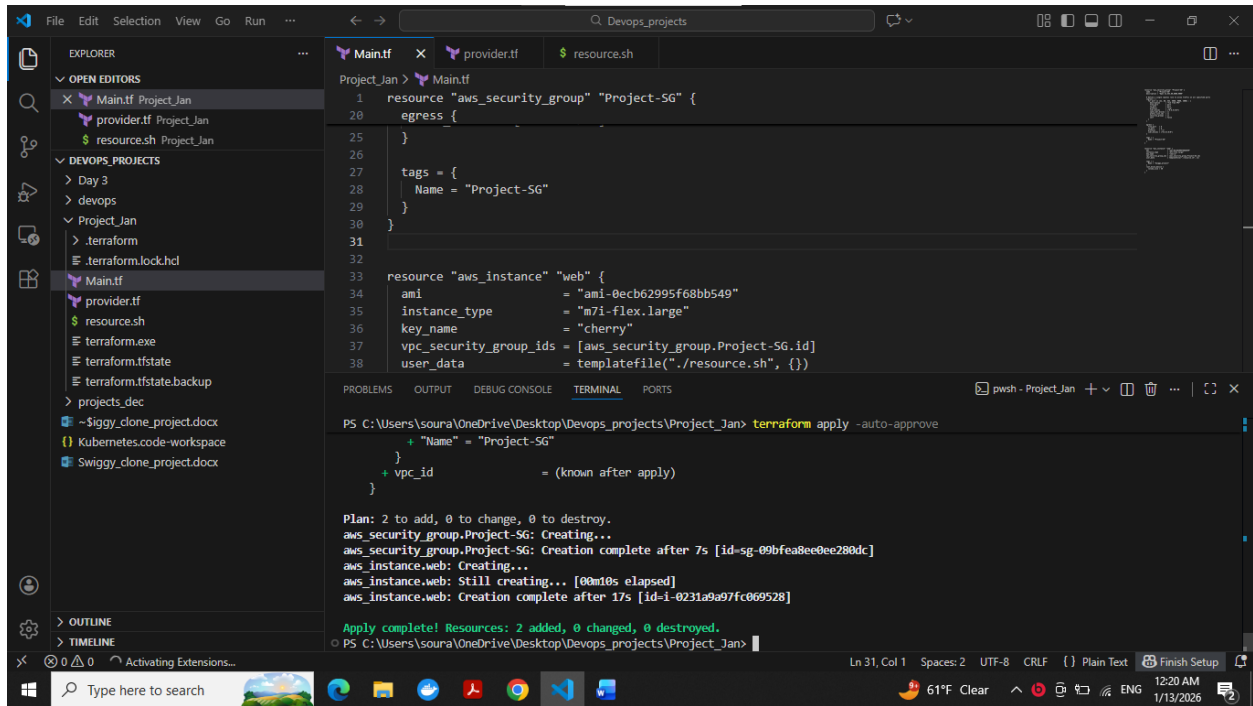
The screenshot shows the Visual Studio Code interface with the Explorer view on the left displaying the project structure. The main editor shows the `Main.tf` file with Terraform configuration for an AWS security group and an instance. The terminal at the bottom shows the execution of the `terraform plan` command, which displays the execution plan for the infrastructure, including the resources to be added, changed, or destroyed.

```
PS C:\Users\soura\OneDrive\Desktop\Devops_projects\Project_Jan> terraform plan

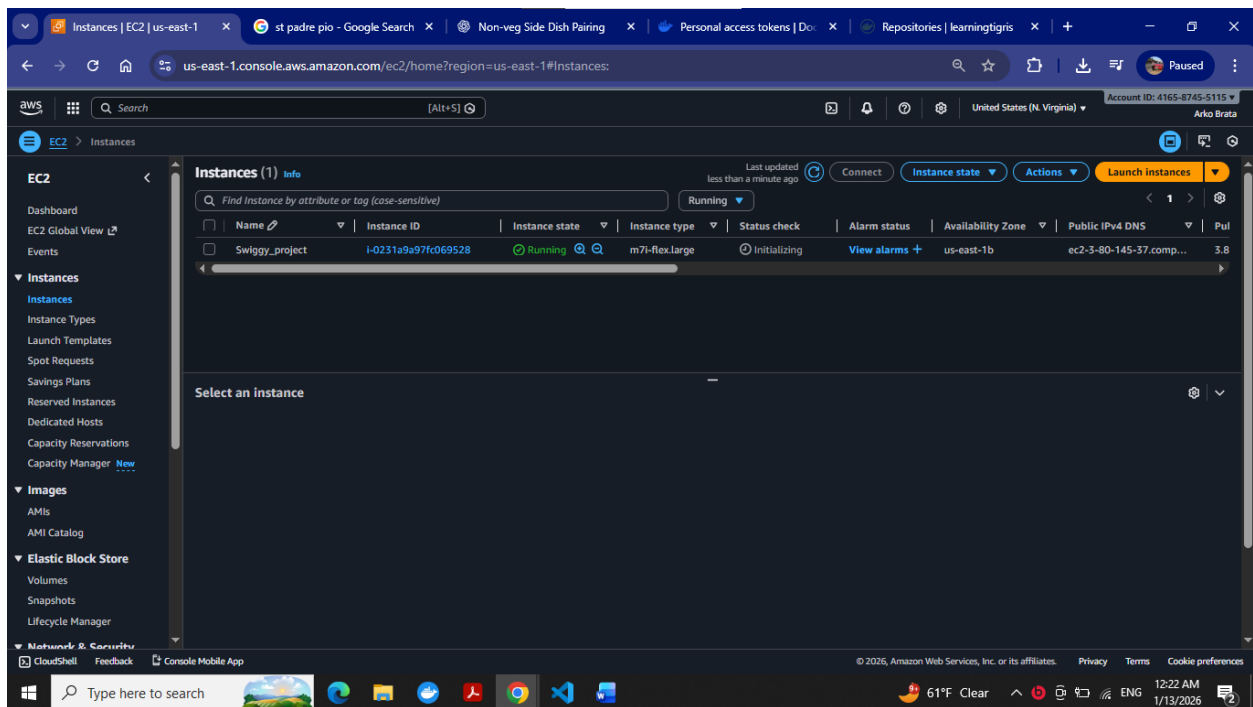
Plan: 2 to add, 0 to change, 0 to destroy.

Note: You didn't use the -out option to save this plan, so Terraform can't guarantee to take exactly these actions if you run "terraform
apply" now.
PS C:\Users\soura\OneDrive\Desktop\Devops_projects\Project_Jan> terraform apply -auto-approve
aws_security_group.Project-SG: Refreshing state... [id=sg-0965d61a2fd99700]
aws_instance.web: Refreshing state... [id=i-08557f417d4ad71ad]
```

terraform apply -auto-approve # Provisions EC2 instance and networking



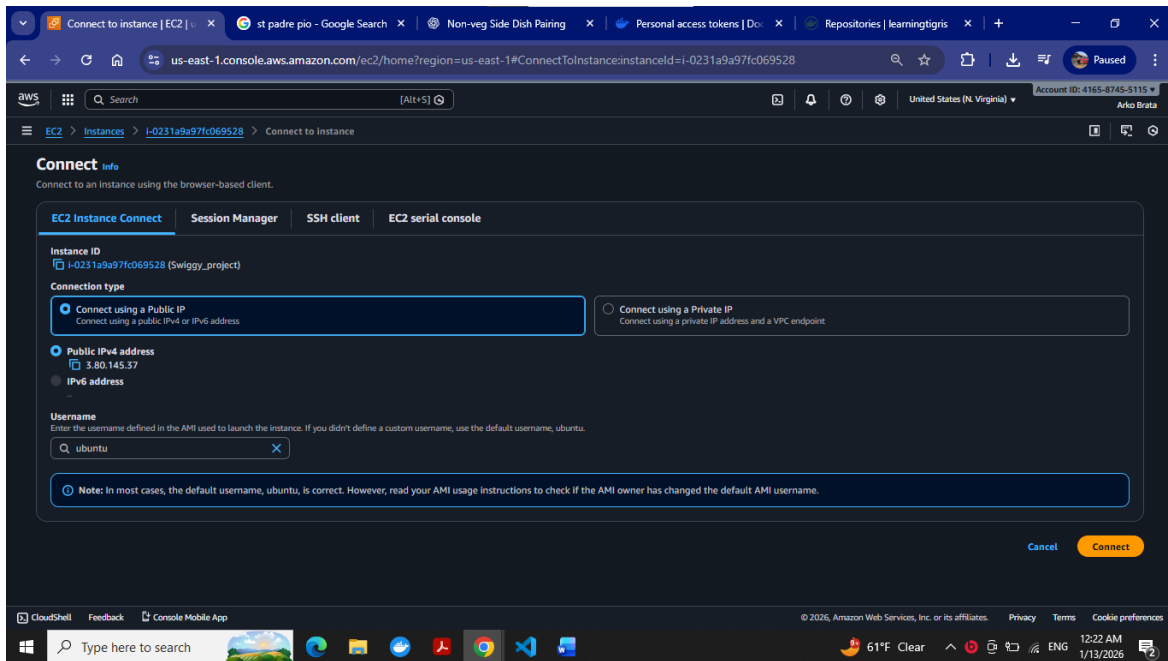
Result: AWS EC2 instance is successfully provisioned and configured.



Step 2: Connect to the EC2 Instance

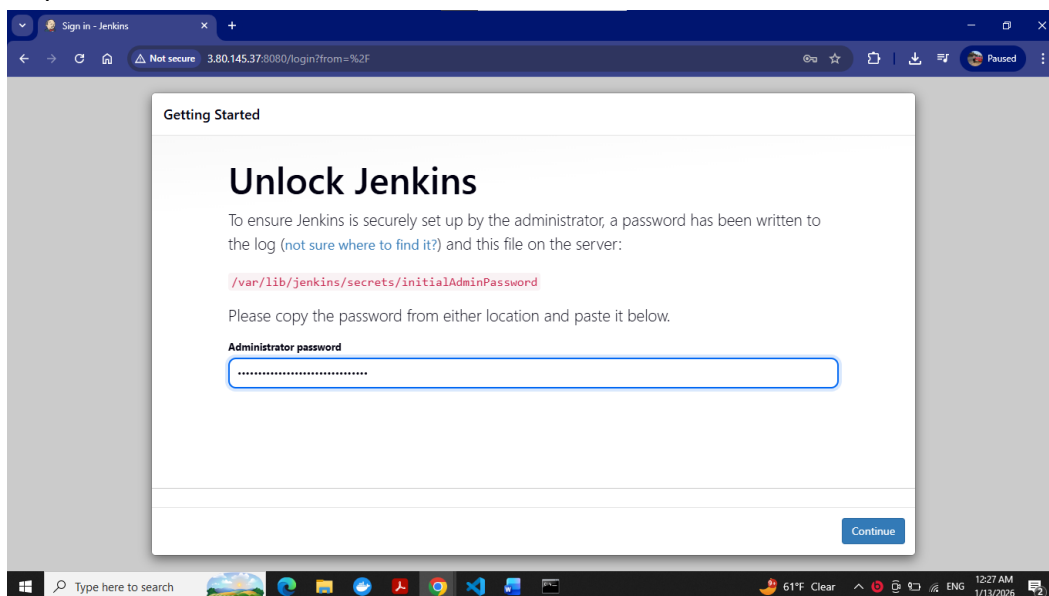
Using EC2 Instance Connect

1. Navigate to **AWS Console** → **EC2** → **Instances**
2. Select the running instance
3. Click **Connect** → **EC2 Instance Connect**



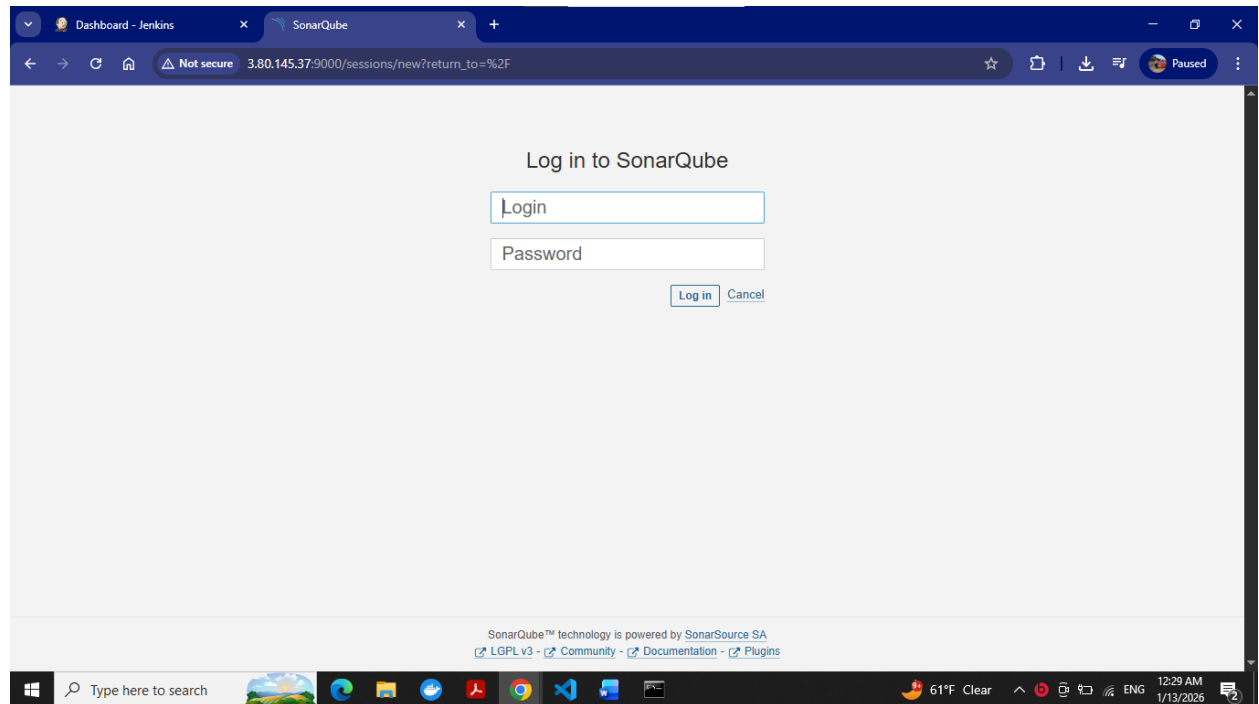
Verify Installed Services

- **Jenkins:**
`http://<EC2-Public IP>:8080`



- **SonarQube:**

http://<EC2- Public IP>:9000

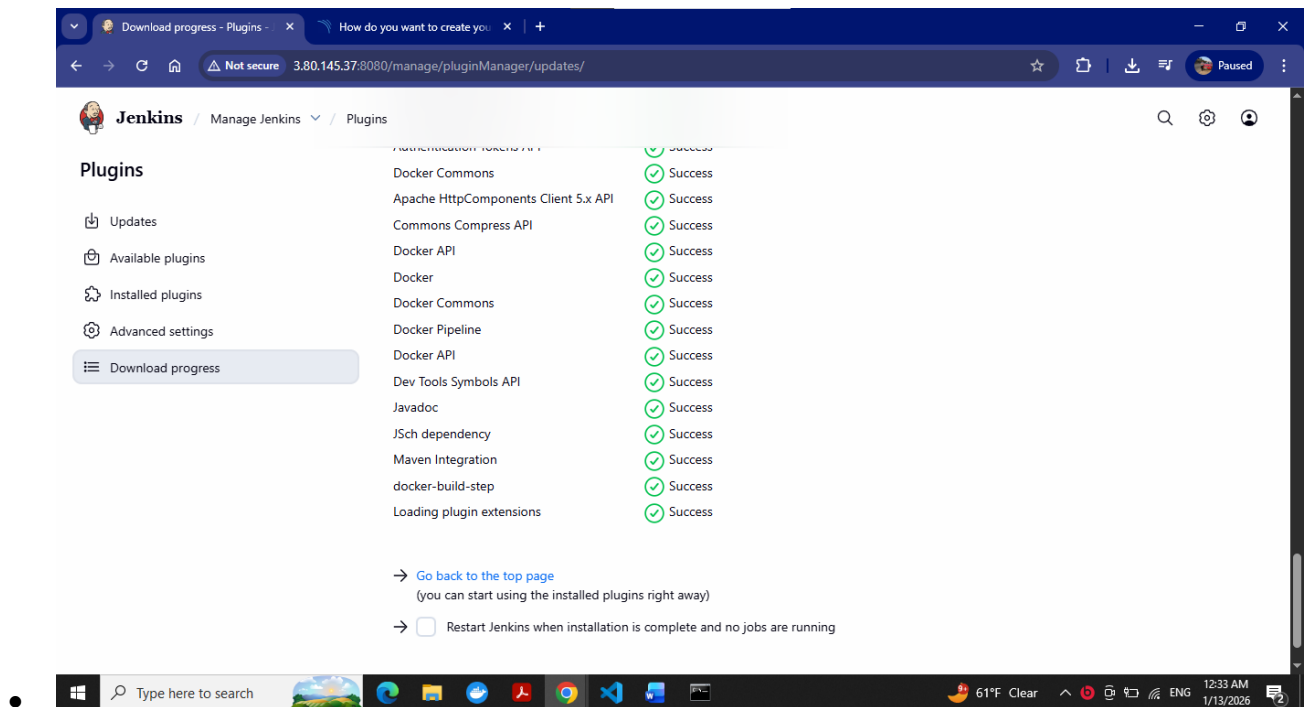


Step 3: Jenkins Plugin and Tool Configuration

Required Jenkins Plugins:

Installed via **Manage Jenkins** → **Plugins** → **Available**:

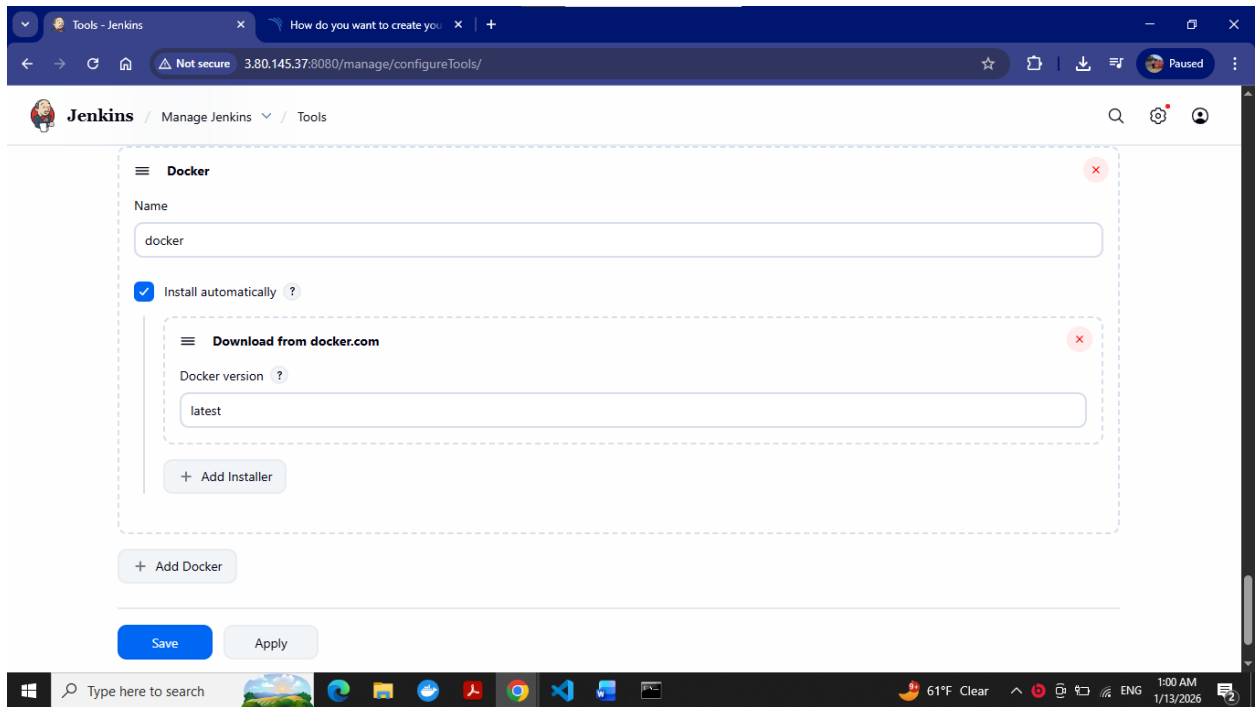
- **Eclipse Temurin (JDK)** – Required for Java-based tools like Jenkins and SonarQube Scanner.
- **Pipeline Stage View** – Provides a visual representation of pipeline stages.
- **SonarQube Scanner** – Enables Jenkins to perform static code analysis.
- **NodeJS** – Required for building and running Node.js applications.
- **Docker (Common, Pipeline, API, Build-Step)** – Enables Docker image build, push, and container management.



Global Tool Configuration:

Configured under **Manage Jenkins** → **Global Tool Configuration**:

- **JDK → jdk17 (from adoptium.net)**
Needed because Jenkins, SonarQube scanner, and many build tools depend on Java. Java 17(v17.0.11+9) is stable and widely supported.
- **SonarQube Scanner → sonar-scanner (v8.0.1.6346)**
This is the CLI used by Jenkins to run static code analysis and send results to the SonarQube server.
- **NodeJS → node23 (v23.0.0)**
Required to install and run JavaScript/Node dependencies (npm install) for the Swiggy app.
- **Docker → latest**
Enables Jenkins to build Docker images, tag them, and push to DockerHub. It's also used to run the containerized application.

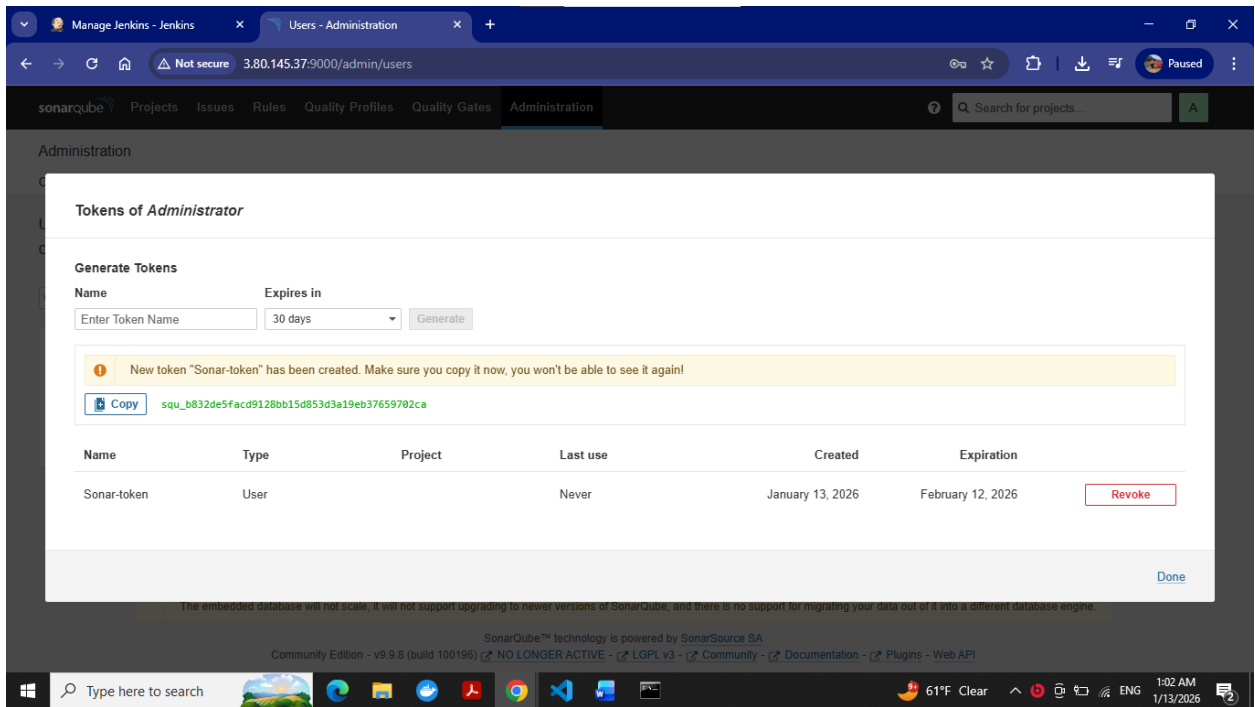


These tools are used by Jenkins pipelines during build, analysis, and deployment stages.

Step 4: SonarQube Integration with Jenkins

Generate a SonarQube Authentication Token:

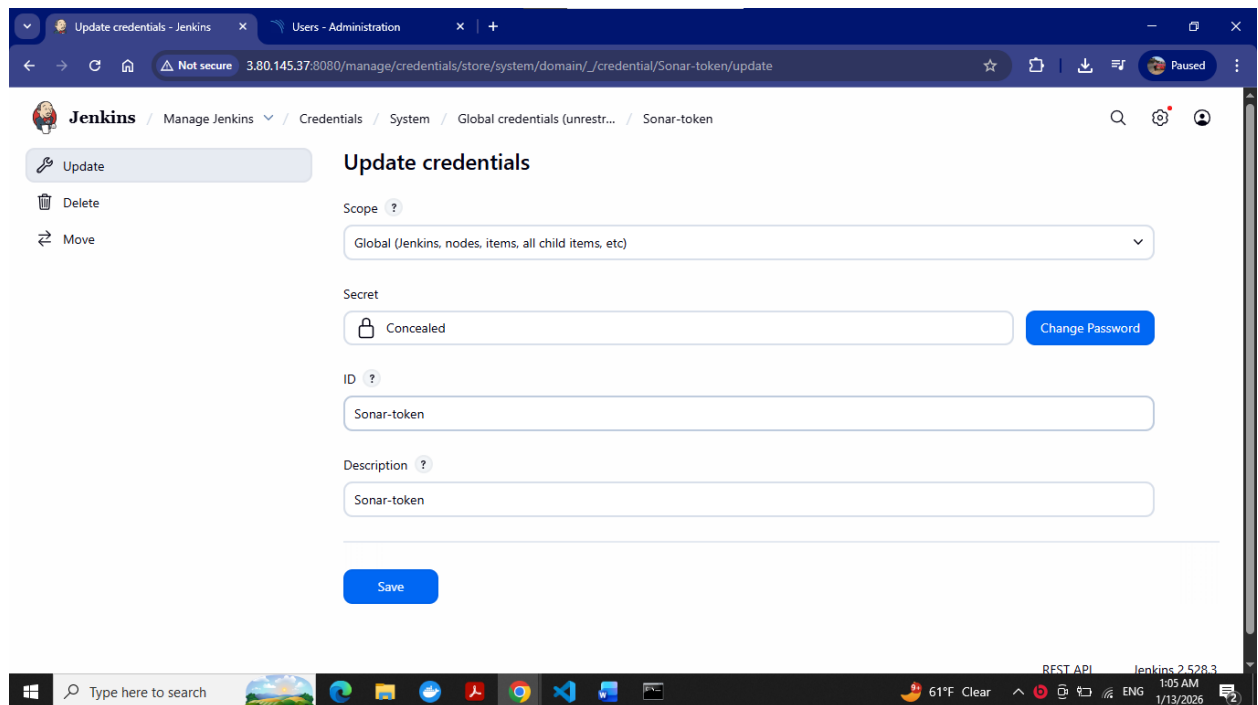
- Go to **SonarQube** → **Administration** → **Security** → **Users**.
- Generate a token named **sonar-token**.



Add SonarQube Token to Jenkins

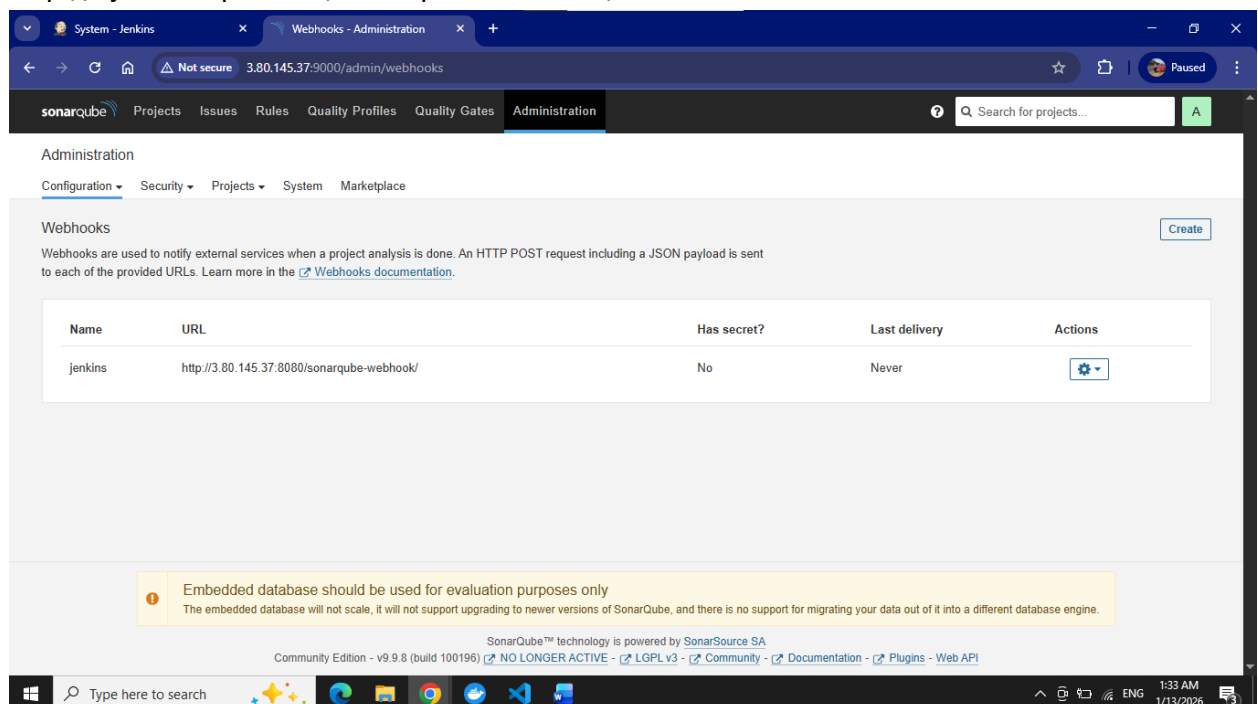
- Navigate to **Manage Jenkins** → **Credentials** → **Global** → **Add Credentials**
- Select **Secret Text**.
- Paste the sonar-token.
- Set the ID as sonar-token.

Now Jenkins can securely communicate with SonarQube.



Configure SonarQube Webhook:

- Go to **SonarQube** → **Administration** → **Configuration** → **Webhooks**.
- Add webhook URL:
`http://<jenkins-ip>:8080/sonarqube-webhook/`



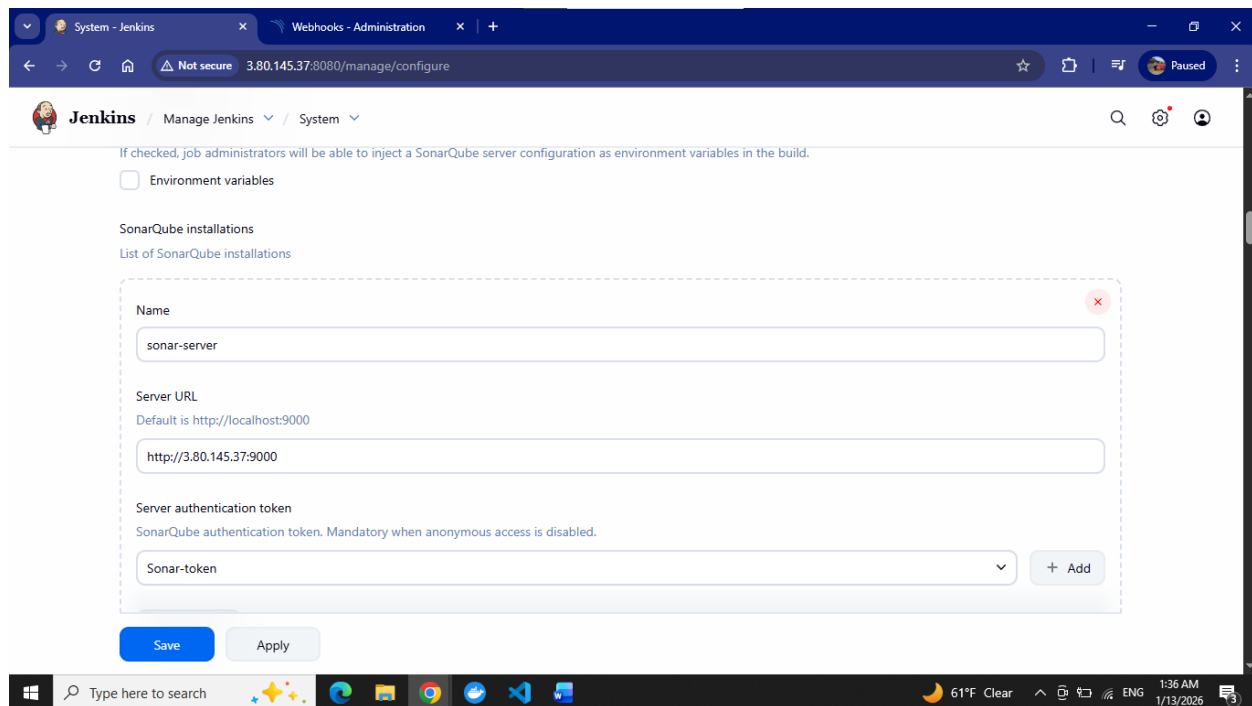
- This webhook notifies Jenkins of the scan results (so the Quality Gate stage can continue or fail accordingly).

Configure SonarQube Server in Jenkins:

- Go to **Manage Jenkins → Configure System → SonarQube Servers**.
- Configuration:
 - **Name:** sonar-server
 - **URL:** http://<EC2-Public IP>:9000
 - **Authentication Token:** sonar-token

This enables pipelines to use:

withSonarQubeEnv('sonar-server')

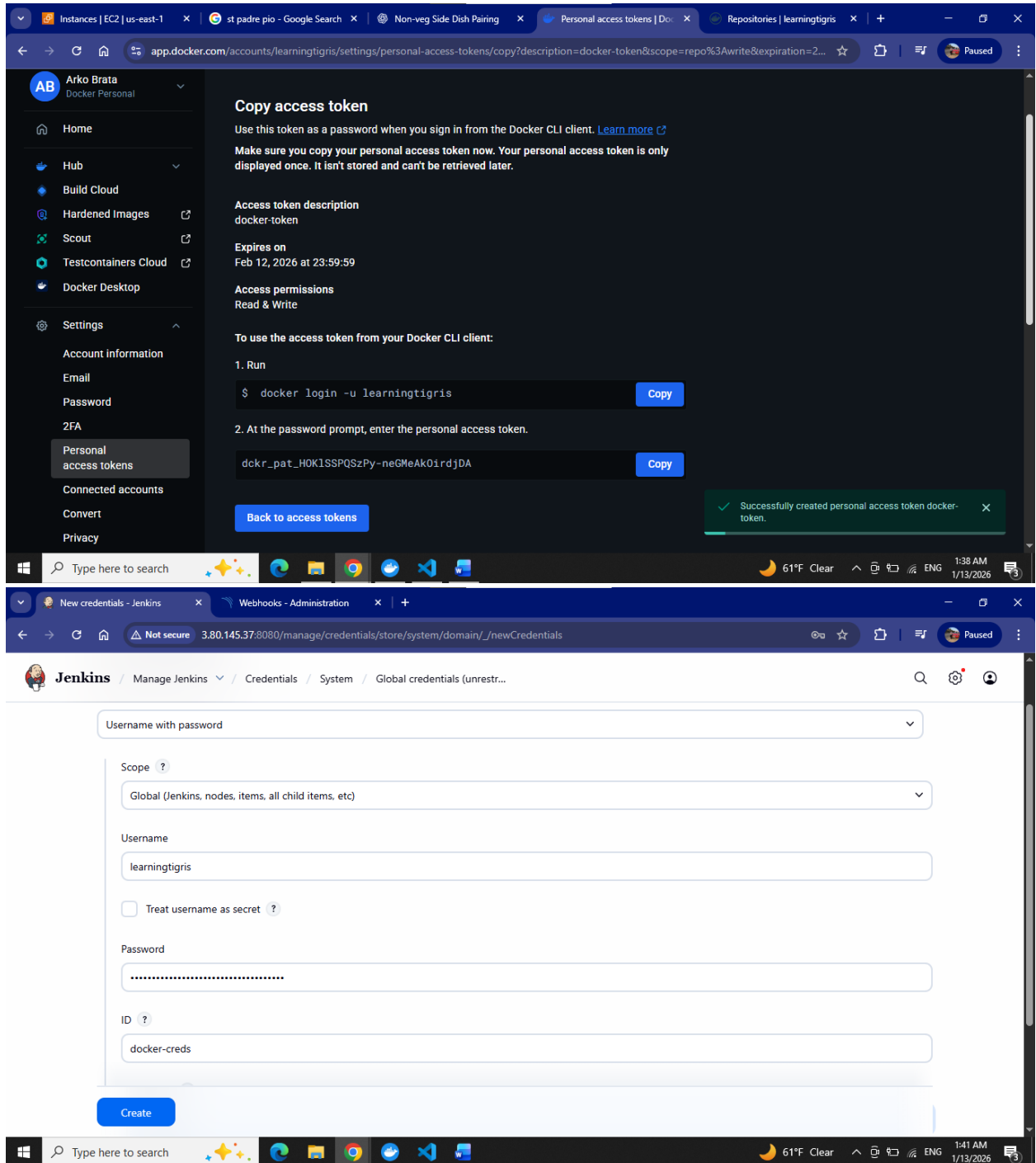


Step 5: DockerHub Credential Configuration

To push Docker images securely:

- Go to **Manage Jenkins → Credentials → Global → Add Credentials**
- Choose **Username with password**

- Provide:
 - DockerHub username
 - DockerHub password or access token
- Set ID as: docker-creds



The image shows two screenshots from a Windows desktop. The top screenshot is from the Docker Hub website, specifically the 'Copy access token' page for the user 'Arko Brata'. It displays the access token description, expiration date (Feb 12, 2026), and permissions (Read & Write). It also provides instructions on how to use the token with the Docker CLI, showing the command `docker login -u learningtigris` and the token itself: `dcKr_pat_HOK1SSPQsZPy~neGMeAk0irdjDA`. A success message at the bottom states 'Successfully created personal access token docker-token.' The bottom screenshot is from the Jenkins 'New credentials' page. It shows the configuration for a 'Username with password' credential. The scope is set to 'Global (Jenkins, nodes, items, all child items, etc)'. The username is 'learningtigris' and the ID is 'docker-creds'. The password field is masked with dots. A 'Create' button is visible at the bottom.

Docker Hub Screenshot:

- Copy access token**
- Use this token as a password when you sign in from the Docker CLI client. [Learn more](#)
- Make sure you copy your personal access token now. Your personal access token is only displayed once. It isn't stored and can't be retrieved later.
- Access token description:** docker-token
- Expires on:** Feb 12, 2026 at 23:59:59
- Access permissions:** Read & Write
- To use the access token from your Docker CLI client:**
 1. Run
`$ docker login -u learningtigris` [Copy](#)
 2. At the password prompt, enter the personal access token.
`dcKr_pat_HOK1SSPQsZPy~neGMeAk0irdjDA` [Copy](#)
- [Back to access tokens](#)
- Successfully created personal access token docker-token.

Jenkins Screenshot:

- Jenkins** / Manage Jenkins / Credentials / System / Global credentials (unrestr...)
- Username with password
- Scope: Global (Jenkins, nodes, items, all child items, etc)
- Username: learningtigris
- ☐ Treat username as secret
- Password:
- ID: docker-creds
- [Create](#)

Used in pipeline:

withDockerRegistry(credentialsId: 'docker-creds', toolName: 'docker')

This allows Jenkins to:

- Build Docker images
- Tag images
- Push images to DockerHub securely

Step 6: Create Jenkins Pipeline Job

Application Source Code

GitHub Repository: <https://github.com/learningtigris/Project-Swiggy.git>

Pipeline Creation

- Go to **Jenkins → New Item**
- Select **Pipeline**
- Configure pipeline script

Pipeline Script

```
pipeline{
  agent any
  tools{
    jdk 'jdk17'
    nodejs 'node23'
  }
  environment {
    SCANNER_HOME=tool 'sonar-scanner'
  }
  stages {
    stage('clean workspace'){
      steps{
        cleanWs()
      }
    }
    stage('Checkout from Git'){
      steps{
        git 'https://github.com/learningtigris/Project-Swiggy.git'
      }
    }
  }
}
```

```

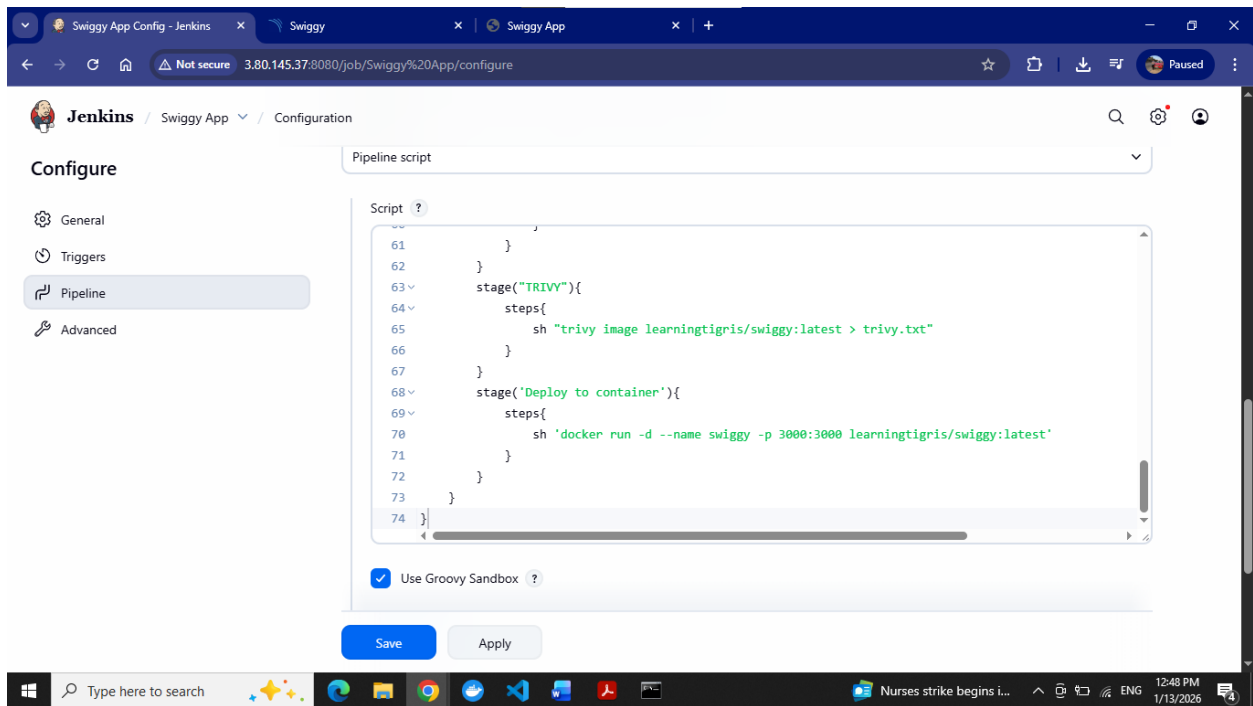
}
stage("Sonarqube Analysis "){
  steps{
    withSonarQubeEnv('sonar-server') {
      sh "' $SCANNER_HOME/bin/sonar-scanner -Dsonar.projectName=Swiggy \
        -Dsonar.projectKey=Swiggy '"
    }
  }
}
stage("quality gate"){
  steps {
    script {
      waitForQualityGate abortPipeline: false, credentialsId: 'Sonar-token'
    }
  }
}
stage('Install Dependencies') {
  steps {
    sh "npm install"
  }
}
stage('TRIVY FS SCAN') {
  steps {
    sh "trivy fs . > trivyfs.txt"
  }
}
stage("Docker Build & Push"){
  steps{
    script{
      withDockerRegistry(credentialsId: 'docker-creds', toolName: 'docker'){
        sh "docker build -t swiggy ."
        sh "docker tag swiggy learningtigris/swiggy:latest "
        sh "docker push learningtigris/swiggy:latest "
      }
    }
  }
}
stage("TRIVY"){

```

```

steps{
    sh "trivy image learningtigris/swiggy:latest > trivy.txt"
}
}
stage('Deploy to container'){
    steps{
        sh 'docker run -d --name swiggy -p 3000:3000 learningtigris/swiggy:latest'
    }
}
}
}
}

```



Step 7: Build and Deploy Application

Pipeline Execution Flow

- Source Code Checkout from GitHub
- SonarQube Code Analysis
- Quality Gate Validation
- Trivy File System Security Scan
- Docker Image Build
- Trivy Docker Image Vulnerability Scan
- Push Image to DockerHub
- Deploy Docker Container on EC2

The screenshot displays the Jenkins web interface for a pipeline named 'Swiggy'. The left sidebar contains navigation links: Status, Changes, Build Now, Configure, Delete Pipeline, Full Stage View, SonarQube, Stages, Rename, and Pipeline Syntax. The main area shows the 'View' tab with a table of stage execution times. Above the table, the average stage times are listed: Declarative: Tool Install (180ms), clean workspace (332ms), Checkout from Git (719ms), Sonarqube Analysis (13s), quality gate (348ms), Install Dependencies (11s), TRIVY FS SCAN (1s), Docker Build & Push (1min 40s), TRIVY (1min 42s), and Deploy to container (667ms). The table below shows the execution times for three builds. The first two builds are successful, while the third build failed at the 'Deploy to container' stage.

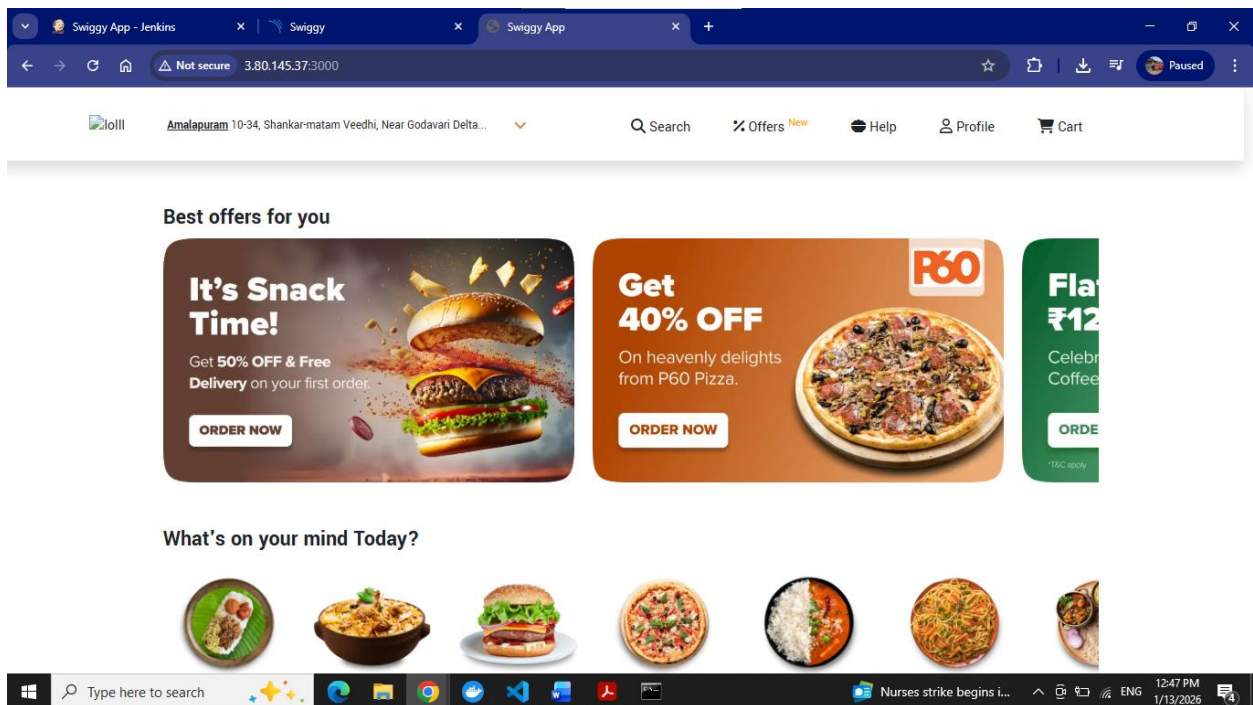
	Declarative: Tool Install	clean workspace	Checkout from Git	Sonarqube Analysis	quality gate	Install Dependencies	TRIVY FS SCAN	Docker Build & Push	TRIVY	Deploy to container
Average stage times: (full run time: ~3min 44s)	180ms	332ms	719ms	13s	348ms	11s	1s	1min 40s	1min 42s	667ms
No Changes	174ms	342ms	768ms	13s	366ms (paused for 1s)	11s	1s	1min 19s	1min 53s	779ms
No Changes	187ms	342ms	811ms	12s	320ms (paused for 1s)	11s	1s	1min 44s	1min 36s	579ms failed
No Changes	179ms	314ms	579ms	13s	359ms (paused for 919ms)	12s	1s	1min 57s	1min 36s	644ms failed

Below the table, the 'SonarQube Quality Gate' section is visible. The bottom of the screenshot shows the Windows taskbar with the time 4:32 PM on 1/13/2026.

Final Application Verification

<http://<EC2-Public IP>:3000>

Swiggy application is successfully deployed and live.



Key Skills Demonstrated

- Infrastructure as Code (Terraform)
- CI/CD Pipeline Automation (Jenkins)
- Code Quality Enforcement (SonarQube)
- Security Scanning (Trivy)
- Containerization & Deployment (Docker)
- AWS EC2 Administration
- Credential & Secret Management